Base de datos key-value replicada

Programación Concurrente y Paralela · Segundo cuatrimestre de 2026

1. Estado de cada réplica

Cada réplica es un proceso enlazado al invocante de `start_link/2`. Recibe los pids de sus pares y la capacidad α. El estado pasa por los argumentos de su ciclo de recepción.

- `Valores` guarda hasta α asociaciones `Clave => {Version, {valor, Valor, Vence}}`. Cada clave ocupa a lo sumo una plaza.
- `Metadatos` guarda la mayor versión aplicada de cada clave y su estado, `valor`, `borrado` o `desalojado`. Las marcas sobreviven a la salida del valor y permiten rechazar mensajes obsoletos.
- `Recientes` registra claves y versiones de las últimas escrituras conocidas que definen la protección de capacidad. La cantidad de registros está acotada por el cupo protegido.
- `Pendientes` registra cada lectura en curso, su cliente, la clave, los pares que todavía deben contestar y el mejor candidato transitorio. La entrada se elimina al responder. Solo `Valores` se usa como almacenamiento de la base de datos.

2. Versiones, conflictos y borrados

Cada escritura o borrado recibe una versión de `erlang:unique_integer([monotonic])`. Dentro de la única VM BEAM, las versiones son únicas y crecientes. Una réplica compara la versión entrante con la mayor que conoce para esa clave. El máximo es independiente del orden de entrega. Una marca de borrado conserva su versión aunque se haya liberado la plaza. Un `put` anterior queda descartado y uno posterior puede volver a crear la clave.

Un desalojo conserva la versión como marca `desalojado`. Una copia de esa misma versión puede recuperarse por consulta o volver a ocupar una plaza. Esto permite distinguir la falta de una copia local del borrado de la clave.

3. Vencimiento

`put/4` valida el TTL en el invocante. Se mide en milisegundos y admite enteros no negativos e `infinity`. La réplica de origen convierte un TTL finito en un instante del reloj monótono de la VM, y replica ese mismo vencimiento. El TTL cero vence al escribir.

Al contestar un `get`, se evalúa el vencimiento del candidato ganador, incluso si llegó desde un par durante una espera. Un vencido da `not_found`. Al hacer lugar, el TTL sirve también para priorizar una víctima entre los candidatos elegibles. Una escritura protegida conserva su plaza aunque haya vencido.

4. Protocolo de comunicación

Mensaje	Emisor	Receptor	Efecto
`{configurar, Pares, Alfa}`	Invocante de `start_link/2`	Réplica nueva	Inicializa los pares, la capacidad y los mapas.
`{put, Cliente, RefCliente, Clave, Valor, TTL}`	Cliente	Réplica elegida	Genera versión y vencimiento, aplica la escritura con el límite de capacidad y propaga el cambio.
`{delete, Cliente, RefCliente, Clave}`	Cliente	Réplica elegida	Quita el valor, guarda una marca versionada y propaga el borrado.
`{get, Cliente, RefCliente, Clave}`	Cliente	Réplica elegida	Responde a partir de la información local o inicia una consulta a los pares.
`{RefCliente, ok}`	Réplica	Cliente	Completa el `put` o `delete` local.
`{RefCliente, ok, Valor}`	Réplica	Cliente	Completa un `get` con el valor vigente.
`{RefCliente, not_found}`	Réplica	Cliente	Completa un `get` sin valor disponible.
`{replicar, Clave, Entrada}`	Réplica de origen	Todos sus pares	Aplica un valor o borrado versionado según el orden de versiones.
`{ofrecer, Clave, Entrada}`	Réplica que desaloja	Todos sus pares	Da al receptor la opción de conservar un valor vigente.
`{consultar, Solicitante, RefConsulta, Clave}`	Réplica que atiende un `get`	Todos sus pares	Solicita la entrada o marca local.
`{respuesta_consulta, RefConsulta, Par, Resultado}`	Par consultado	Réplica solicitante	Aporta un candidato y completa el turno de ese par para esa lectura.
`stop`	Invocante de `stop/1`	Réplica monitoreada	Termina su ciclo de recepción.

Cada pedido usa una referencia única. `put` y `delete` devuelven `ok` después de aplicar el cambio local, sin esperar confirmaciones de los pares. `stop/1` devuelve `ok` después de recibir los avisos de terminación de todos los pids.

5. Capacidad y desalojo

Se protege un cupo de `ceil(3 × α / 4)` claves distintas por su última escritura conocida en la réplica, incluyendo escrituras remotas. En un `put` o una actualización de un par, el valor entrante se considera al armar el ranking. Una oferta aceptada también registra su versión. Un borrado libera de inmediato la plaza y su escritura previa puede seguir en el ranking histórico. Así, el borrado no vuelve a proteger retroactivamente una clave cuya última copia ya se perdió.

Una sobrescritura reutiliza su plaza y una clave nueva usa una plaza libre. Si el mapa está lleno, se elige entre los valores fuera de la protección, incluido el entrante si es elegible. Se toma primero un vencido, en orden de versión, y en su defecto el valor de menor versión. Se retira la víctima antes de insertar en el mapa. Si la víctima es el entrante, se registra su marca de desalojo sin guardarlo.

El corte del ranking se mantiene o sube a medida que se conocen escrituras. Una versión que salió de la protección solo puede volver a quedar protegida con una escritura más nueva de esa clave. Al haber α residentes más el entrante, existe al menos un candidato fuera del cupo protegido. Las escrituras protegidas que sigan vigentes conservan su copia local tras la actualización.

Antes de descartar un valor vigente, se lo ofrece a todos los pares. Un par puede aceptar una oferta en una plaza libre, en la plaza de la misma clave o reemplazando un vencido elegible. Tramitar una oferta no emite mensajes. Por cada desalojo vigente hay a lo sumo un envío por par y ninguna cadena de ofertas. Si todos lo descartan, se admite la pérdida en el peor caso contemplado por la consigna.

6. Algoritmo de `get` y convergencia

Si la réplica tiene un valor o un borrado, usa esa información. Si desconoce la clave o la tiene desalojada, consulta a todos los pares y vuelve al ciclo normal. Cada par contesta con su estado local y sigue atendiendo. Con la referencia de consulta, la solicitante acepta una respuesta por par, elige el candidato de mayor versión y, a igual versión, prefiere un valor a una marca de desalojo. Al completar el conjunto, vuelve a comparar con su información local y verifica el TTL.

La réplica sigue atendiendo durante la espera y cada consulta entre pares se responde en un solo paso local. Bajo los supuestos de procesos vivos, entrega y ejecución eventual, no se forma una dependencia circular. El candidato en `Pendientes` existe durante ese pedido y solo interviene en su respuesta. Los demás pedidos consultan su propia información local y, si hace falta, inician su propia lectura.

Después de que cesan las escrituras y se procesan las actualizaciones y ofertas, todas las réplicas conocen la mayor versión por clave. Si la versión ganadora es un borrado o está vencida, la lectura da `not_found`. Si una réplica conserva una copia vigente, las que la desalojaron pueden obtenerla por consulta. Si se perdió la última copia, todas devuelven `not_found`. Los mapas locales pueden diferir según el momento del desalojo. Las lecturas a lados distintos del instante de expiración pueden observar resultados distintos.

Las pruebas incluyen versiones en órdenes opuestos, borrados atrasados, ofertas, consultas concurrentes, TTL durante una consulta y desalojo asimétrico entre dos réplicas. Verifican escenarios concretos. Los argumentos anteriores cubren el diseño bajo los supuestos de la consigna.

7. Ejemplos de lectura y desalojo

Lectura de un valor desalojado

Dos réplicas tienen capacidad α = 4 y protegen las tres escrituras más recientes. P1 escribe, en orden, `k`, `temporal`, `a`, `b` y `c`. Solo `temporal` tiene TTL finito. P2 está demorada.

Al guardar `c`, P1 protege `a`, `b` y `c`. Los candidatos son `k` y `temporal`, ambos vigentes en ese momento. P1 ofrece y desaloja `k` por ser el más antiguo. Cuando P2 procesa esa misma escritura, `temporal` ya venció. P2 desaloja `temporal` y conserva `k`.

Réplica	Metadatos de `k`	Copia local de `k`	Respuesta de `get/2` para `k` tras la propagación
P1	`{Vk, desalojado}`	Ausente	`{ok, k}`, tras consultar a P2.
P2	`{Vk, valor}`	Vigente, versión `Vk`	`{ok, k}`, desde su copia local.

P1 envía `{consultar, P1, RefConsulta, k}` a P2 y registra el pedido en `Pendientes`. P2 devuelve la entrada de versión `Vk`. Como esa versión coincide con la marca de desalojo de P1, gana la entrada con valor. P1 vuelve a comparar con su estado local, comprueba el TTL, responde al cliente y elimina la consulta. Para `temporal`, P1 detecta el vencimiento al leer su copia. P2 consulta a P1 y comprueba el mismo vencimiento. Ambos devuelven `not_found`.

Si intervienen dos pares, el primer candidato puede quedar en `Pendientes` mientras llega la segunda respuesta. En el test `valor_remoto_vence_antes_de_finalizar_get_test/0`, ese candidato vence durante la espera. Cuando llega la última respuesta, la réplica comprueba el vencimiento y devuelve `not_found`. El candidato se retiene para esa lectura y se elimina al cerrarla. Las plazas de α se contabilizan en `Valores`.

Vencimientos en la frontera del 75 %

Con α = 5 se protegen cuatro escrituras. Una réplica escribe `a`, `b`, `c`, `d` y `e`, en ese orden. `b` y `c` tienen TTL cero. Después recibe dos escrituras remotas más nuevas, `f` y `g`.

Momento	Protegidas	Víctima	Claves que ocupan plaza
Después de `e`	`b, c, d, e`	Ninguna	`a, b, c, d, e`
Llega `f`	`c, d, e, f`	`b`, vencida y elegible	`a, c, d, e, f`
Llega `g`	`d, e, f, g`	`c`, vencida y elegible	`a, d, e, f, g`

Entre `f` y `g`, `c` sigue ocupando una plaza protegida y `get(P, c)` devuelve `not_found`. En ambos desalojos se conserva `a`, que es la escritura más antigua, porque la prioridad entre los elegibles la tiene una vencida.